



NGUYÊN LÝ HỆ ĐIỀU HÀNH

Phần 8: Bộ nhớ ảo

NGUYỄN THỊ HẬU

Khoa Công Nghệ Thông Tin
Đại học Công Nghệ - Đại học quốc gia Hà Nội

Phần 8: Bộ nhớ ảo

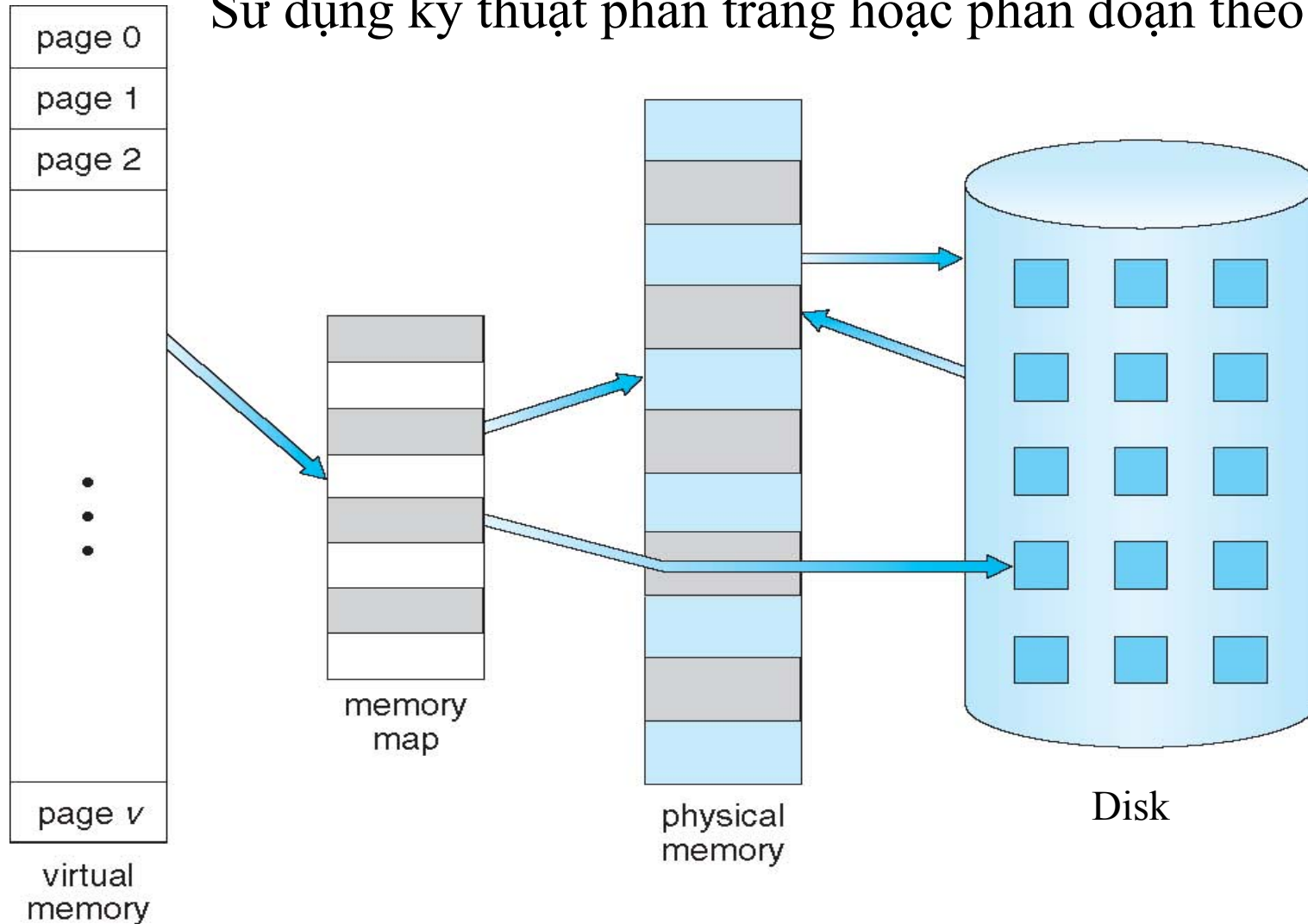
1. Giới thiệu
2. Phân trang theo yêu cầu
3. Sao chép khi viết
4. Thay thế trang
5. Cấp phát frame
6. Thrashing
7. Tập tin ảnh xạ bộ nhớ
8. Cấp phát bộ nhớ cho HĐH

Phần 8: Bộ nhớ ảo

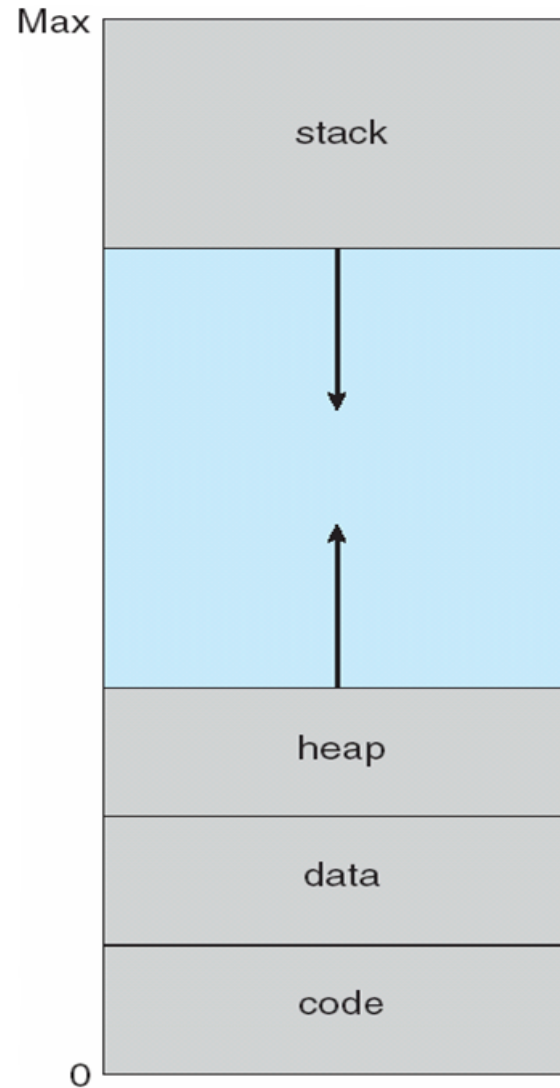
8.1 Giới thiệu

Khái niệm bộ nhớ ảo

Sử dụng kỹ thuật phân trang hoặc phân đoạn theo yêu cầu



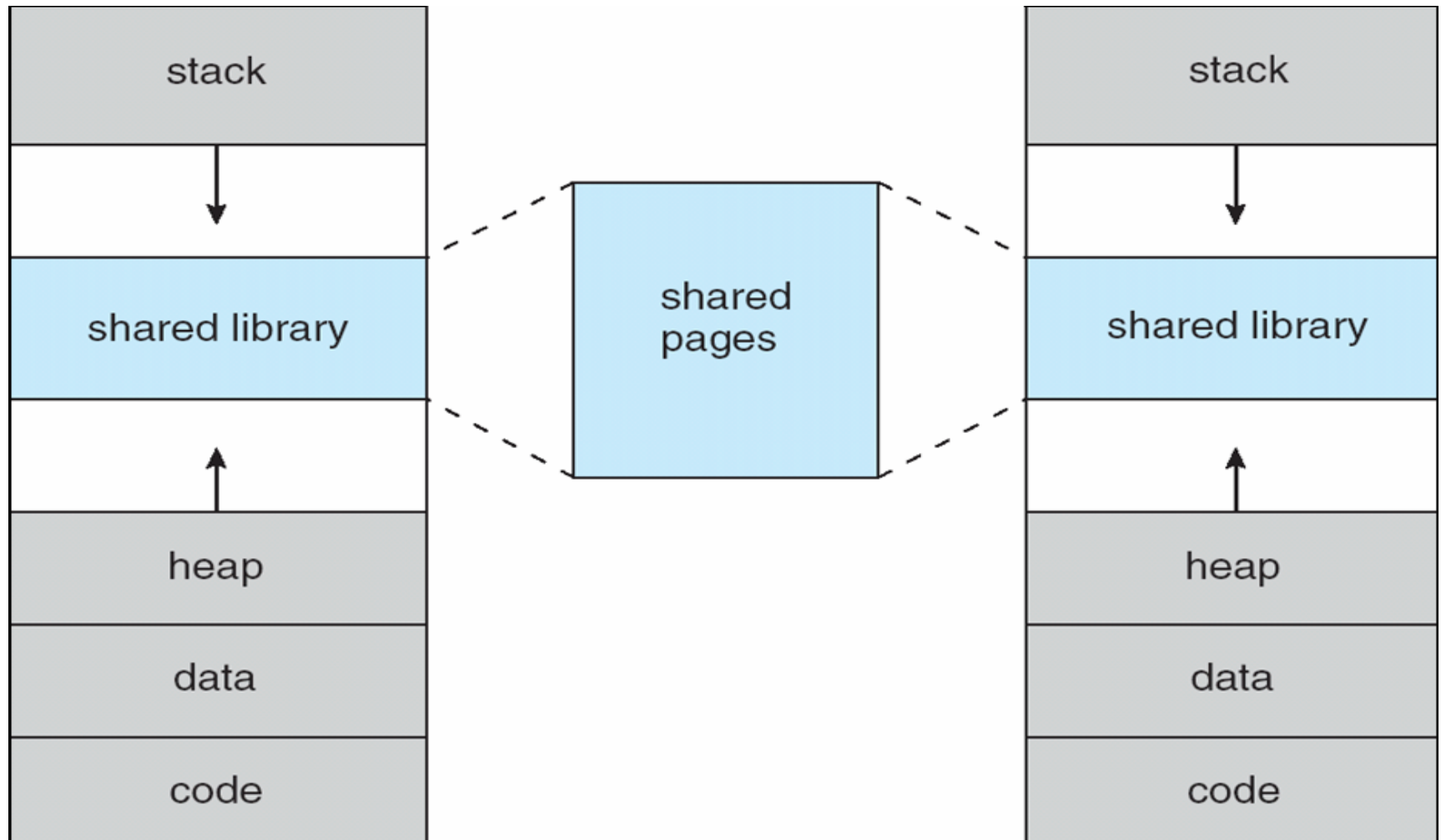
Không gian địa chỉ ảo



→ Liên kết với bộ nhớ vật lý khi cần

Chia sẻ thư viện thông qua bộ nhớ ảo

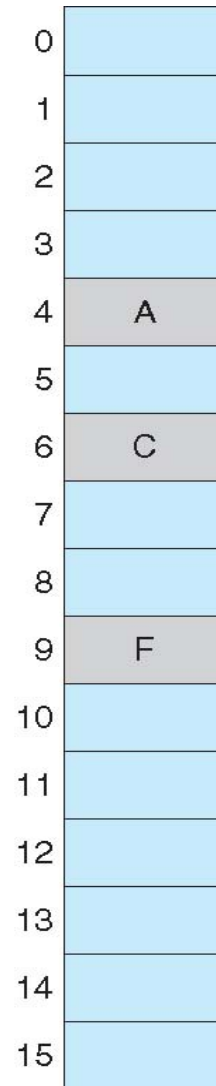
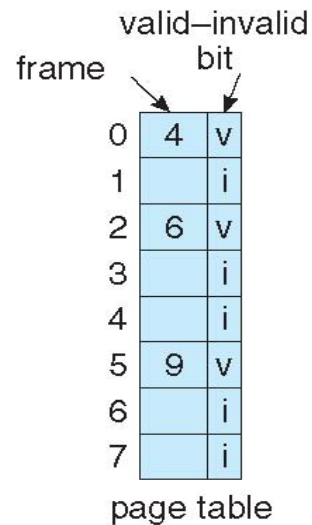
6



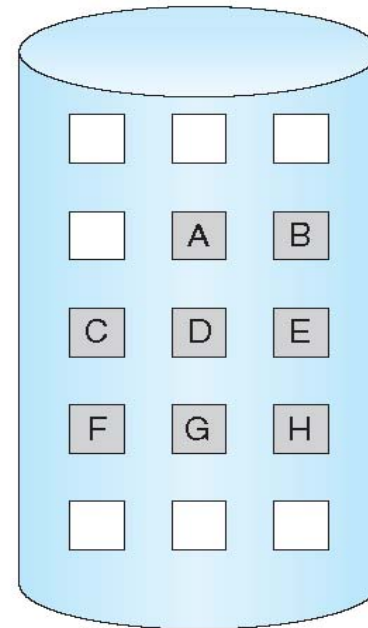
Phần 8: Bộ nhớ ảo

8.2 Phân trang theo yêu cầu

Ví dụ bảng phân trang

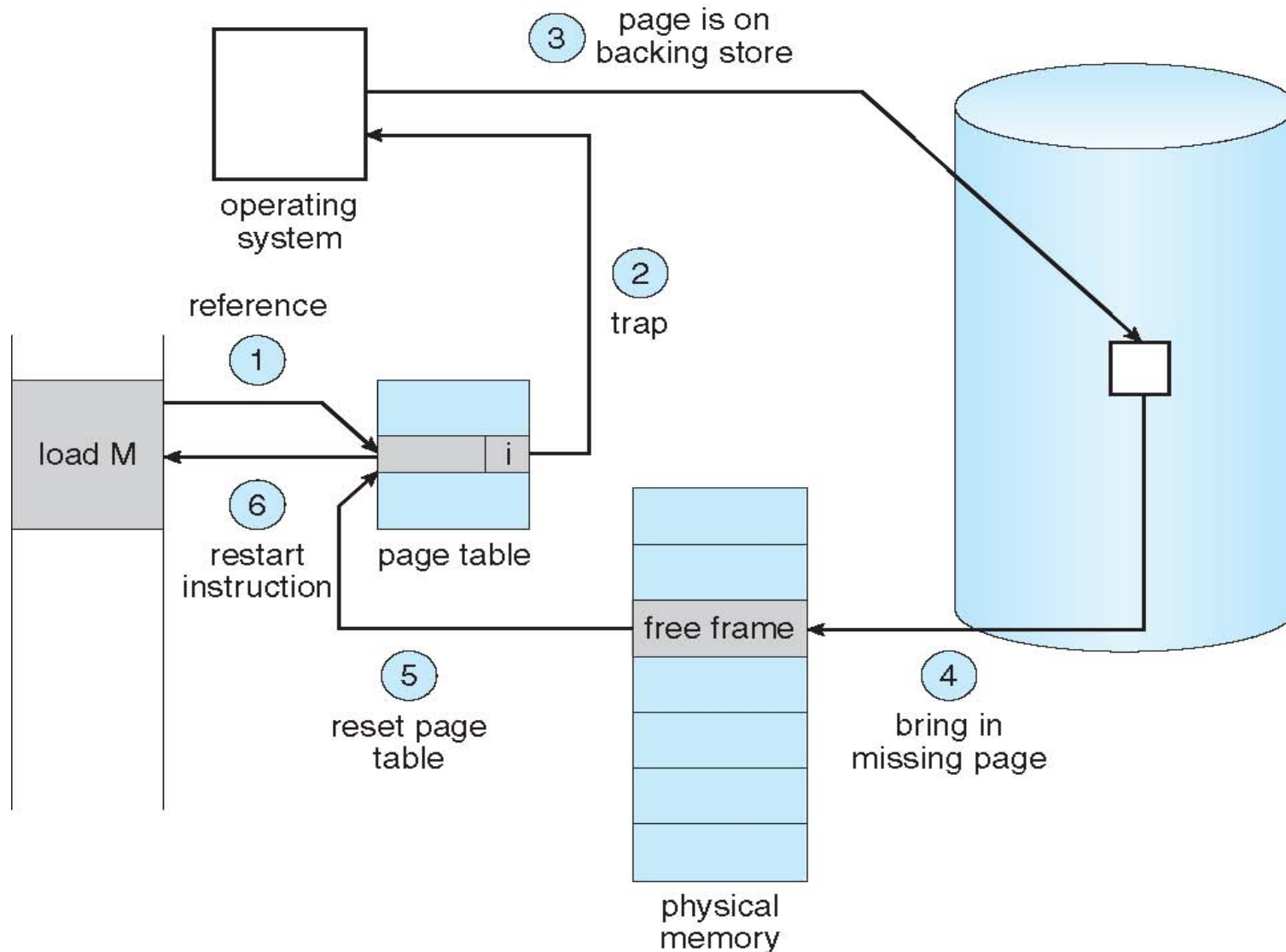


physical memory



lazy swapper: Chỉ tải trang vào bộ nhớ khi cần thiết

Các bước xử lý khi trang chưa được tải vào bộ nhớ trong



Thời gian truy cập hiệu quả

- **m** : thời gian truy cập vào bộ nhớ
- **p** : xác suất một trang chưa được tải vào trong bộ nhớ (**page fault**), $0 \leq p \leq 1$
- *Thời gian truy cập hiệu quả (EAT) = $(1 - p) \cdot m$
+ $p \cdot$ (thời gian phát sinh do page fault
+ chuyển trang ra
+ chuyển trang vào
+ khởi động lại tiến trình)*

Thời gian truy cập hiệu quả

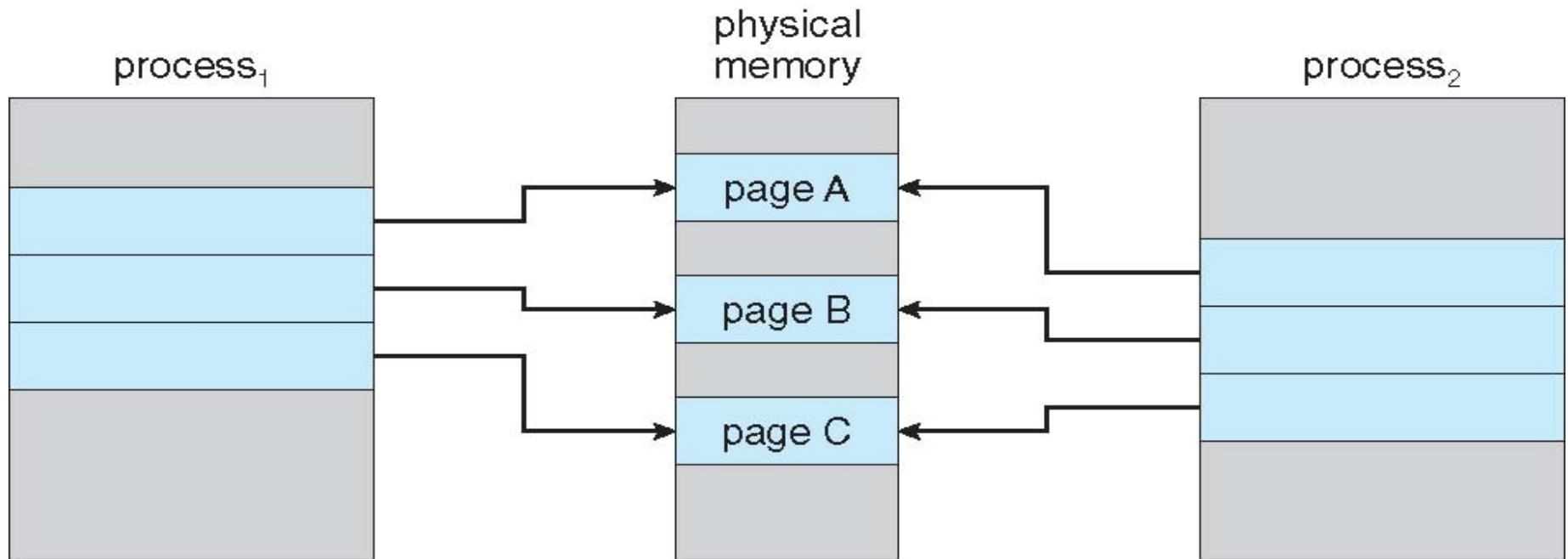
- $m_a = 200 \text{ ns}$
 - Thời gian trung bình của các dịch vụ khi xảy ra page fault: 8 ms
 - $EAT = (1 - p) \cdot 200 + p \cdot 8\,000\,000$
- $$p = 0.001 \quad \rightarrow \quad EAT \sim 8200 \text{ ns} \sim 41. m_a$$

Phần 8: Bộ nhớ ảo

8.3 Sao chép khi viết (Copy on Write)

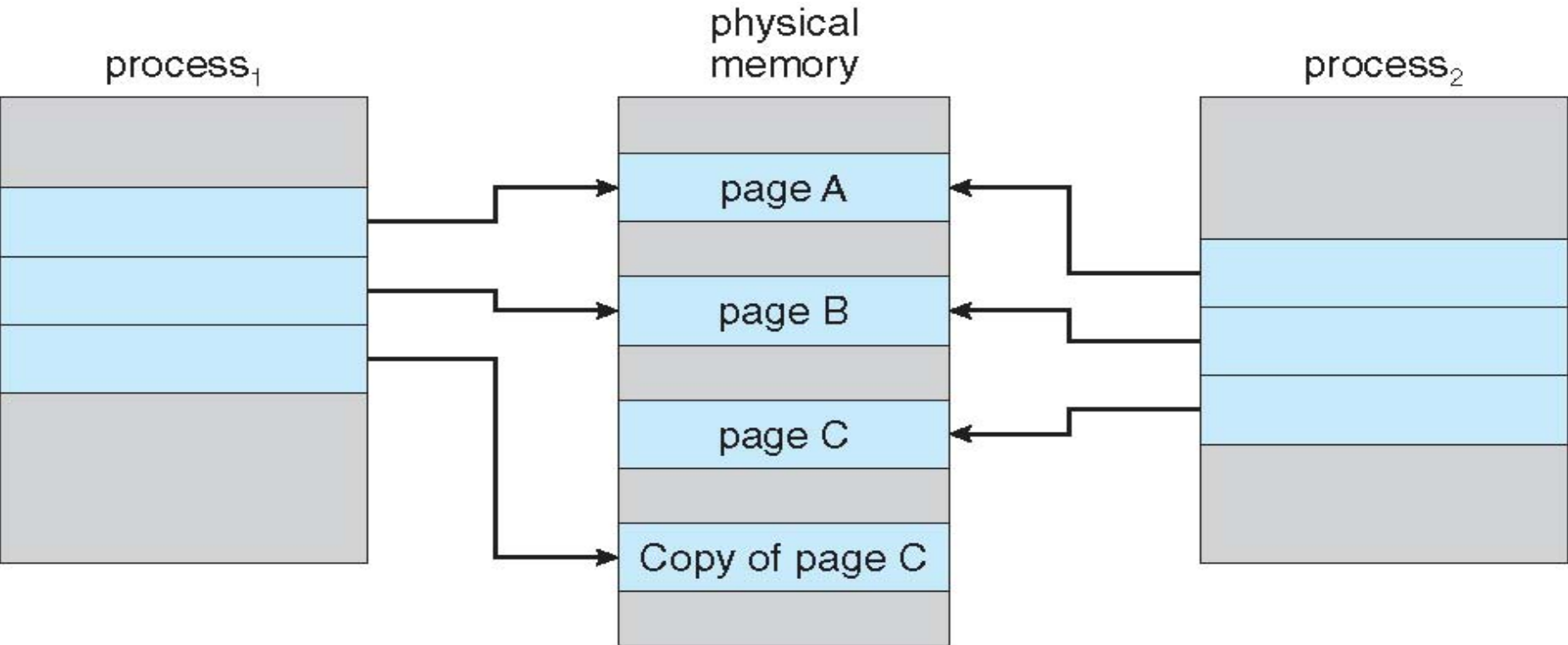
Khái niệm

- Kỹ thuật sao chép khi viết cho phép tiến trình cha và con ban đầu cùng chia sẻ các trang nội dung trong bộ nhớ
 - Nếu một tiến trình thay đổi một trang thì trang đó được sao chép



Ví dụ: Tiến trình 1 thay đổi nội dung trang C

14

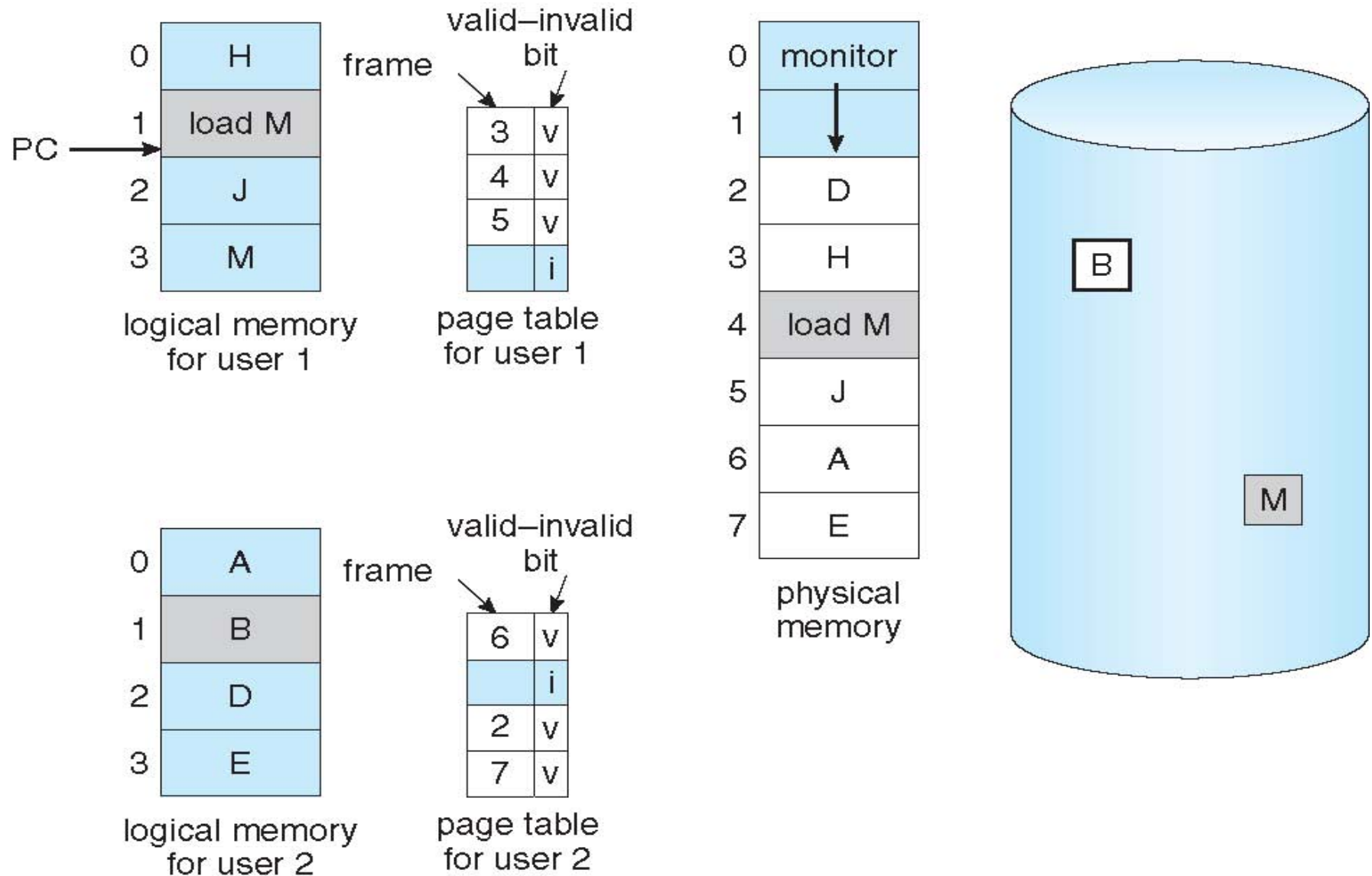


- Cần lựa chọn một trang rỗi trong bộ nhớ
- **Zero-fill-on-demand**: xoá nội dung của trang trước khi thực hiện sao chép

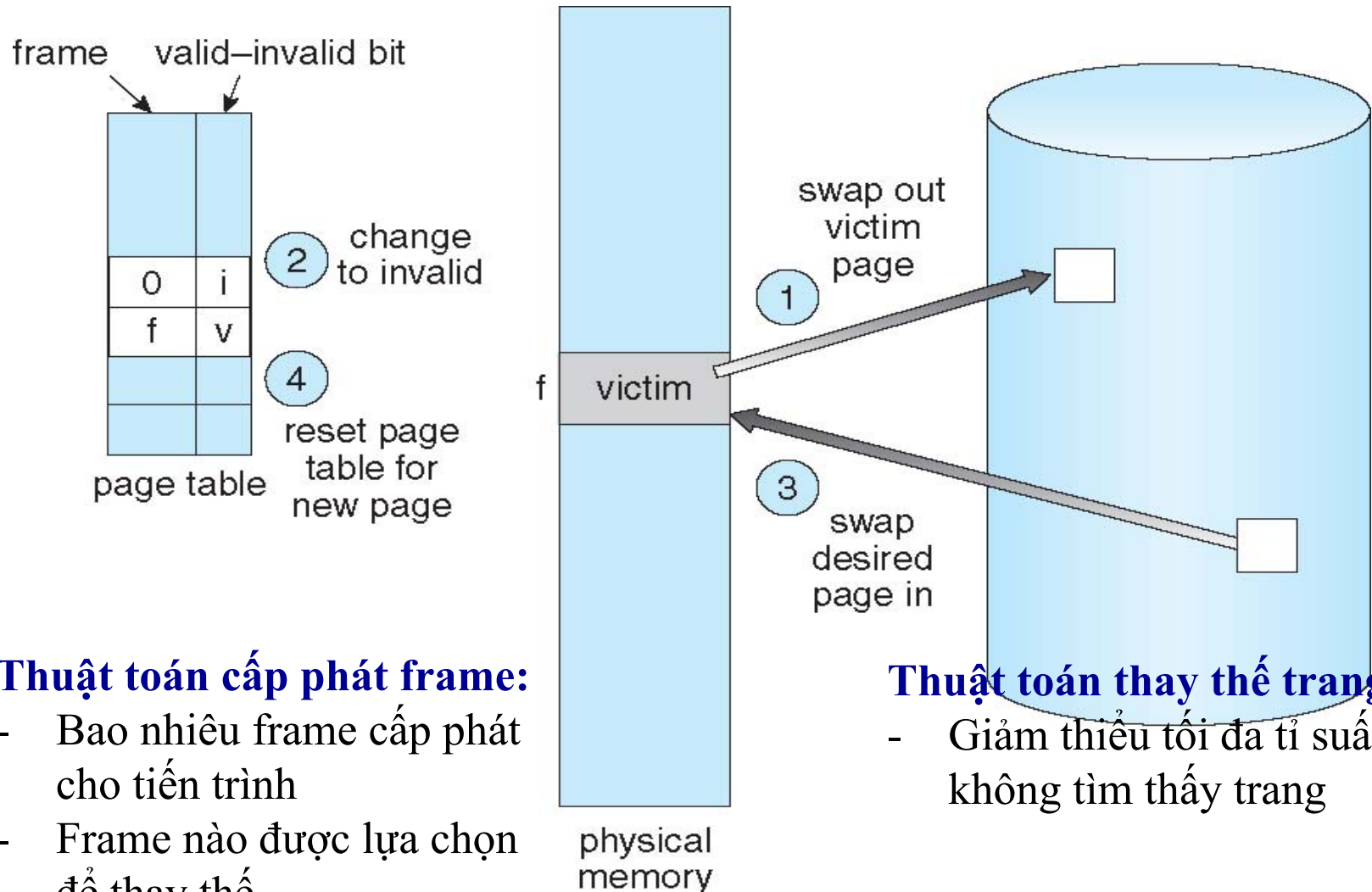
Phần 8: Bộ nhớ ảo

8.4 Thay thế trang

Ví dụ tình huống cần thay thế trang



Quy trình thay thế trang



Thuật toán cấp phát frame:

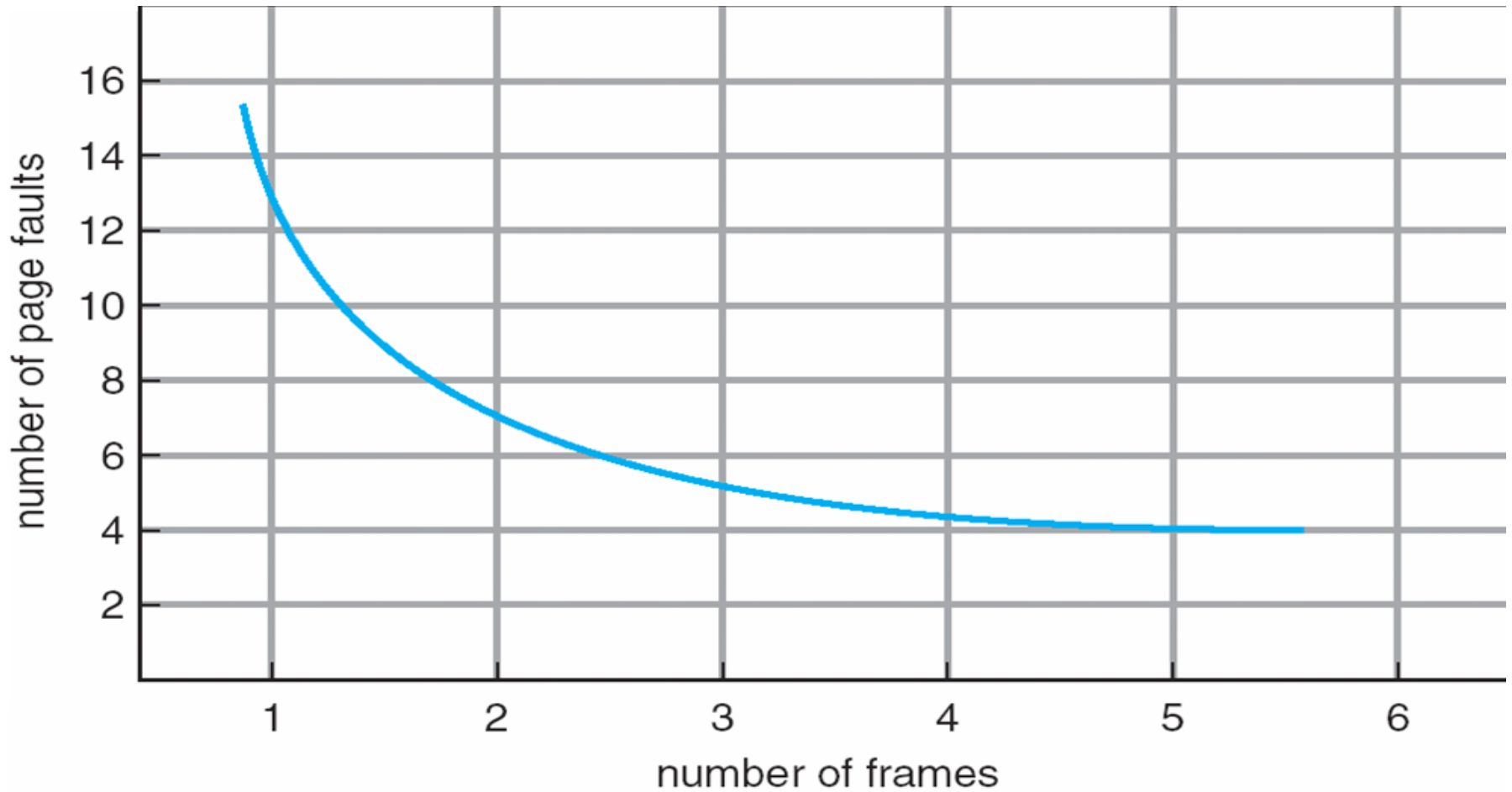
- Bao nhiêu frame cấp phát cho tiến trình
- Frame nào được lựa chọn để thay thế

Thuật toán thay thế trang:

- Giảm thiểu tối đa tỉ suất không tìm thấy trang

Số trang không tìm thấy vs số frame

18

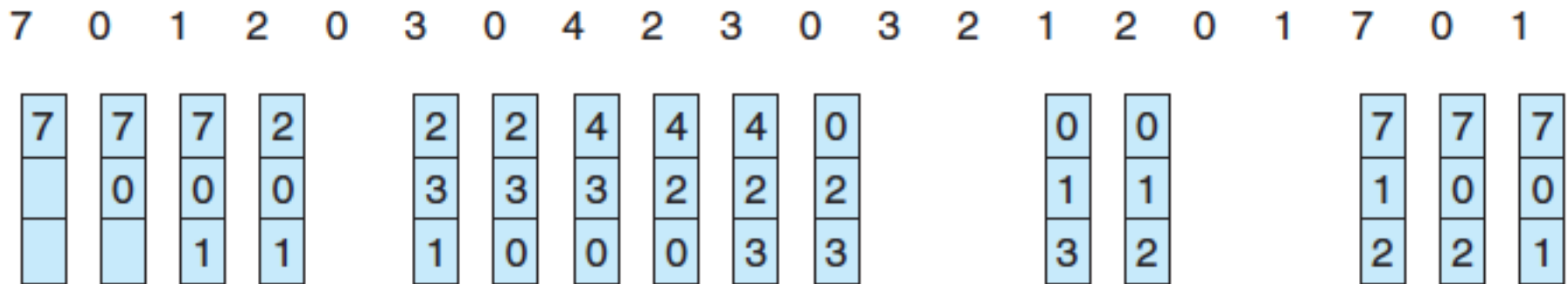


Thuật toán FIFO

- Chuỗi số hiệu các trang cần dùng:

7,0,1,2,0,3,0,4,2,3,0,3,0,3,2,1,2,0,1,7,0,1

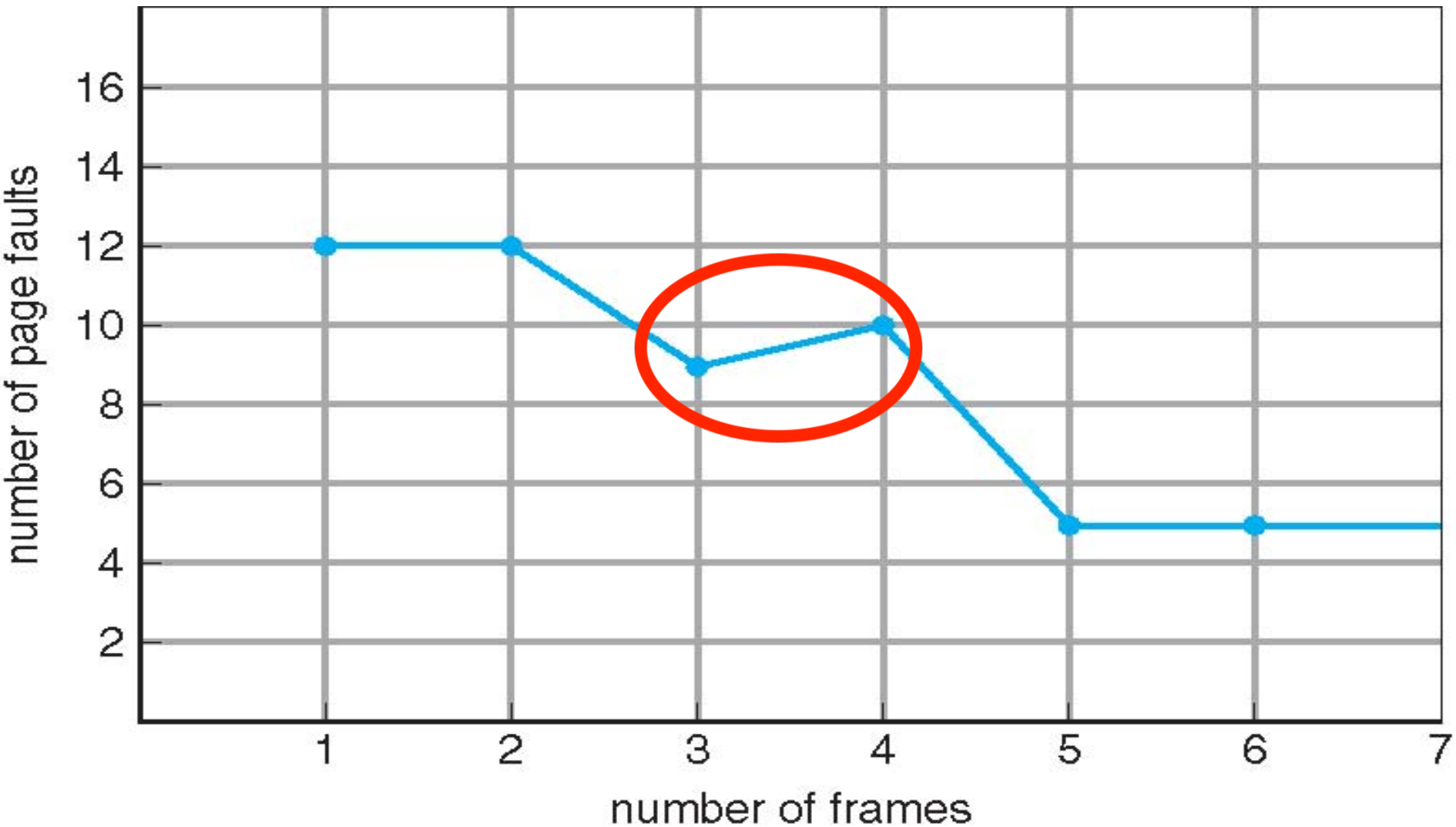
- Bộ nhớ có 3 frame



- 15 page fault

Hiện tượng bất thường Belady với thuật toán FIFO

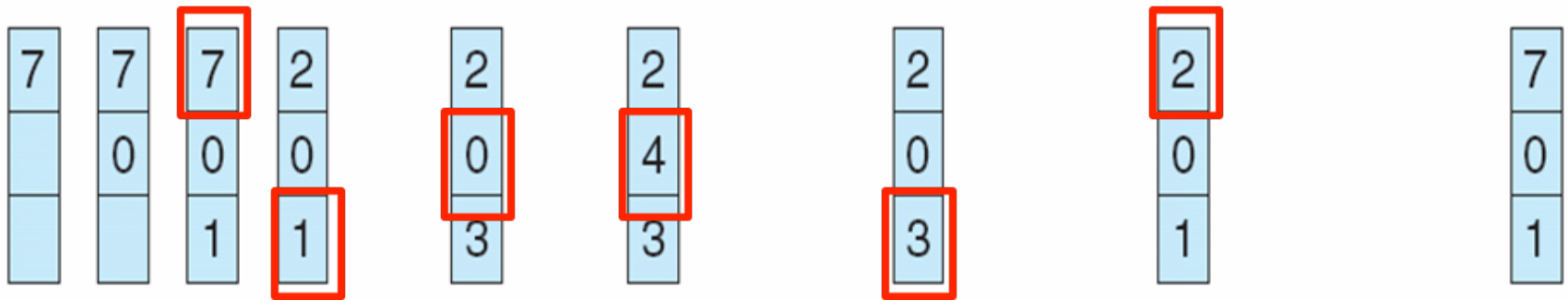
20



Thuật toán tối ưu

- Thay thế trang sẽ không được sử dụng lâu nhất
reference string

7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1



page frames

- Ưu điểm:
 - Tỉ suất page fault thấp nhất
 - Không xảy ra hiện tượng bất thường Belady
- Nhược điểm: Cần biết trước trình tự các trang sẽ được dùng (**khó !**)

Thuật toán thay thế trang đã không sử dụng lâu nhất (LRU)

22

reference string

7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1

7	7	7	2		2		4	4	4	0		1		1		1
	0	0	0		0		0	0	3	3		3		0		0
		1	1		3		3	2	2	2		2		2		7

page frames

- Sử dụng biến đếm hoặc stack
- Không xảy ra hiện tượng bất thường Belady

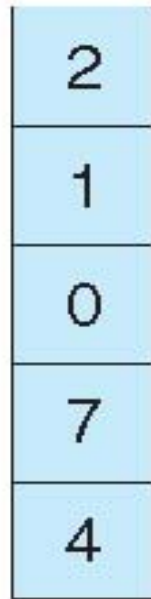
Chú ý: Phân biệt LRU với LFU (tần suất đã sử dụng thấp nhất), và MFU (tần suất đã sử dụng nhiều nhất)

Ví dụ sử dụng stack để ghi lại các trang được dùng gần nhất

23

reference string

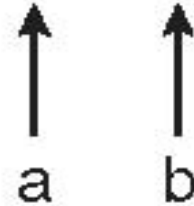
4 7 0 7 1 0 1 2 1 2 7 1 2



stack
before
a



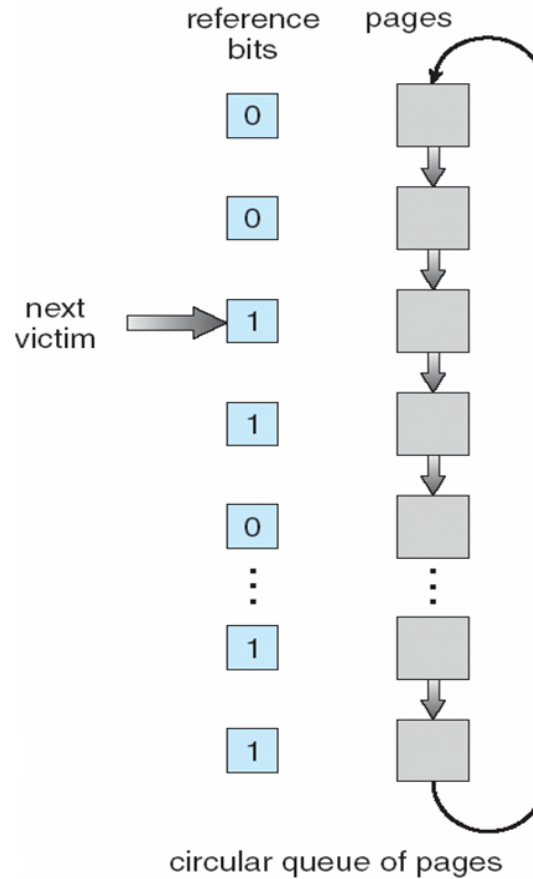
stack
after
b



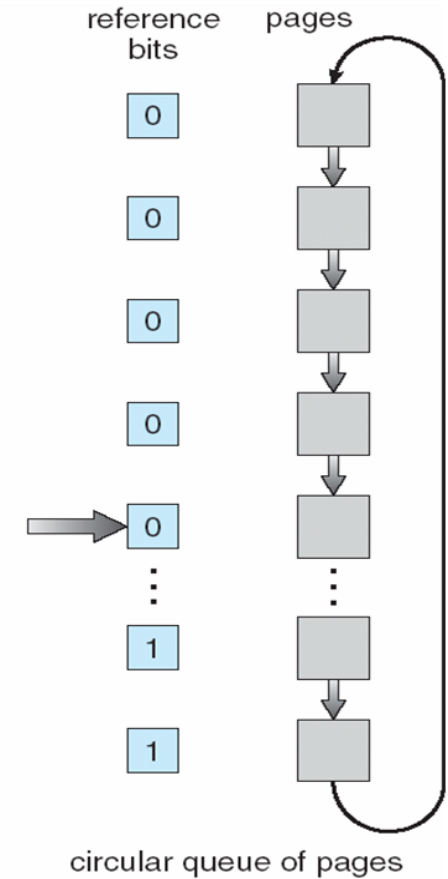
Khi một trang được sử dụng, nó được loại khỏi stack (nếu có), và thêm vào phía trên stack

Thuật toán cơ hội thứ 2

- Lựa chọn trang thay thế theo trình tự FIFO



(a)



(b)

Ref. bit = 1

→ Ref. bit = 0, giữ trang trong bộ nhớ

Chuyển sang trang tiếp theo

Ref. bit = 0 → thay thế

Phần 8: Bộ nhớ ảo

8.5 Cấp phát frame

Cấp phát cố định

- Cấp phát ngang bằng:
 - Các tiến trình được cấp phát số lượng frame giống nhau
- Cấp phát dựa vào kích thước của tiến trình
 - m : số lượng frame
 - s_i : kích thước của tiến trình
 - a_i : số lượng frame cấp phát

Ví dụ: $m=64$, $s_1=10$, $s_2=127 \rightarrow a_1 = (10 \times 64) / 137 \sim 5$

$$a_2 = (10 \times 127) / 137 \sim 59$$

Cấp phát dựa vào trình tự ưu tiên

- Cấp phát số lượng frame cho tiến trình dựa vào trình tự ưu tiên của tiến trình

Cấp phát cục bộ - cấp phát toàn cục

■ Cấp phát toàn cục

- Tiến trình lựa chọn bất cứ frame nào trong bộ nhớ để thay thế → có thể lấy frame từ tiến trình khác

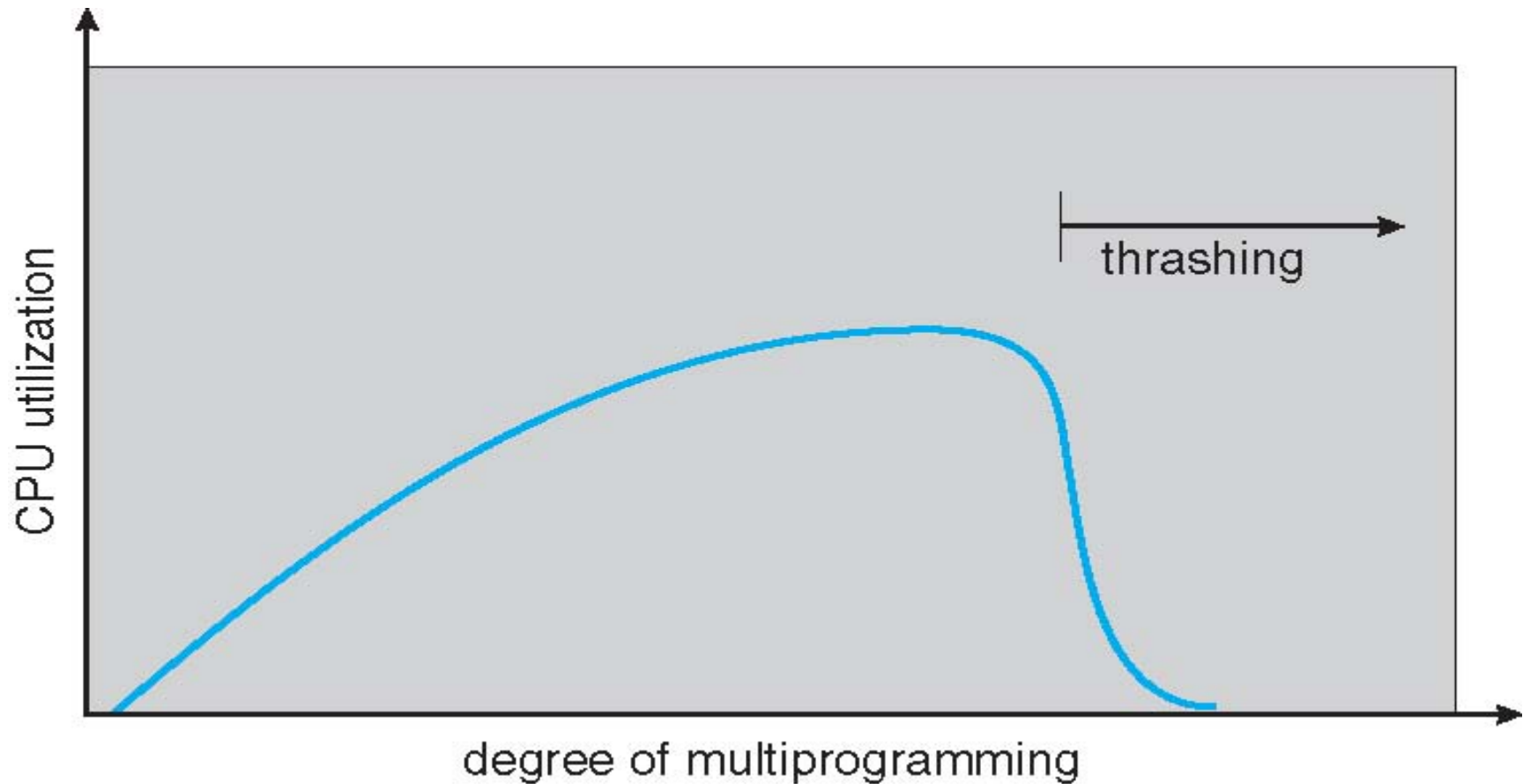
■ Cấp phát cục bộ:

- Chỉ lựa chọn thay thế frame từ những frame đã được phân

Phần 8: Bộ nhớ ảo

8.6 Thrashing

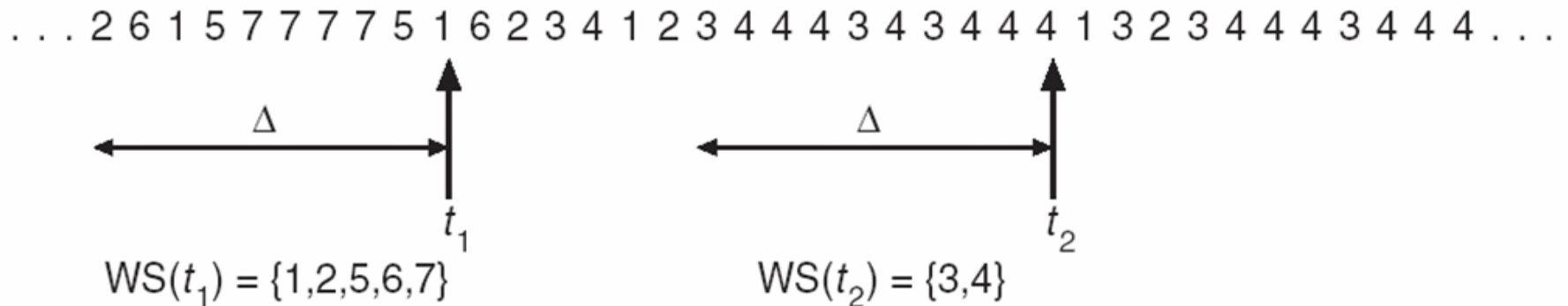
Khái niệm



- Thrashing là hiện tượng tiến trình liên tục chuyển trang ra/vào

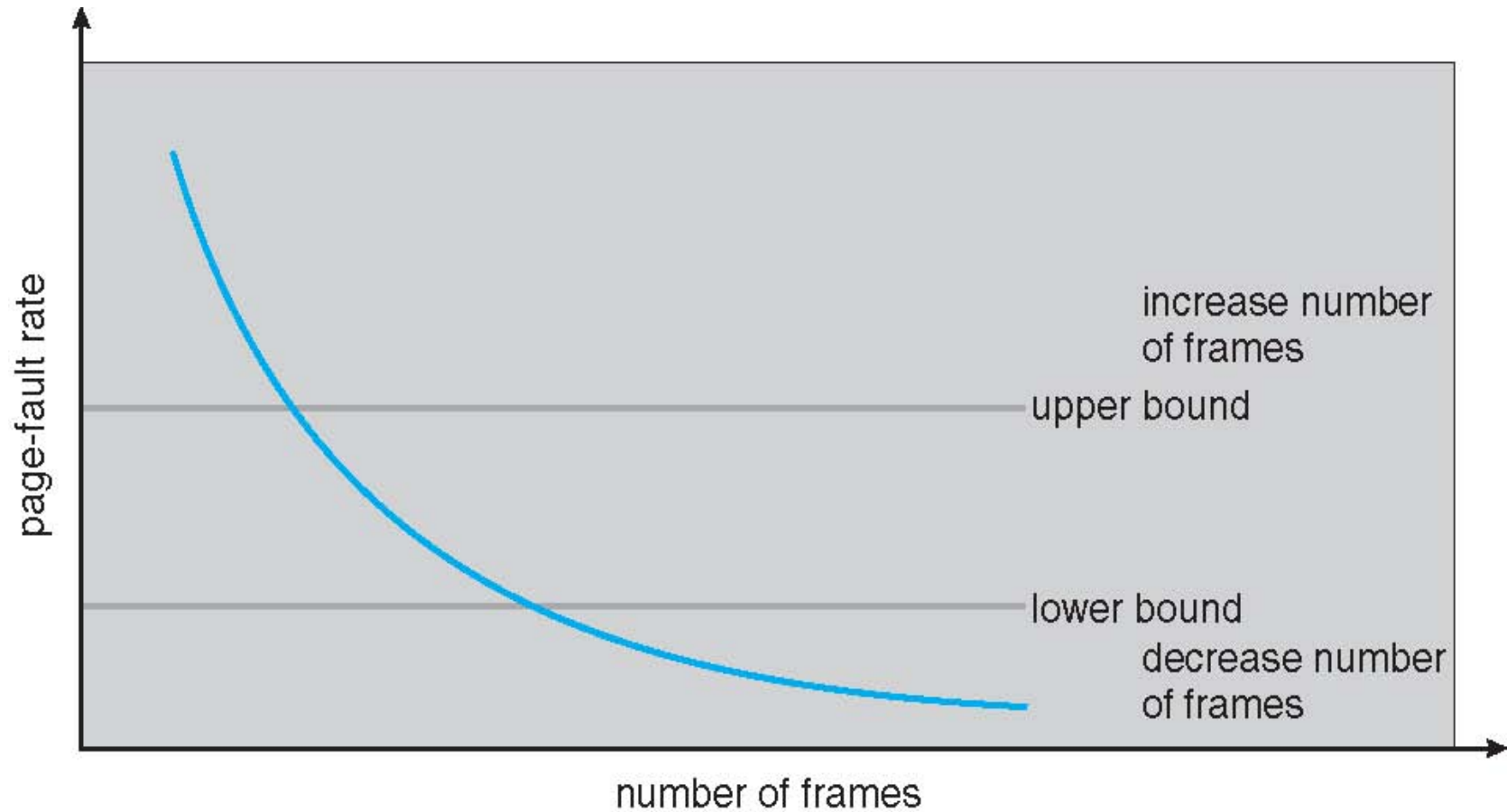
Mô hình tập các trang đang làm việc

- Δ = Cửa sổ tập các trang đang làm việc = số lượng cố định các trang được dùng
 - WSS_i (tập các trang đang làm việc của P_i) = tổng số trang được sử dụng bởi Δ gần nhất
 - $D = \sum WSS_i$ (tổng số frame cần)
 - m : tổng số frame $\rightarrow$ Nếu $D > m$ thì có hiện tượng thrashing
- page reference table



- Cấp phát frame cho các trang trong Δ

Phương pháp sử dụng tần suất page fault

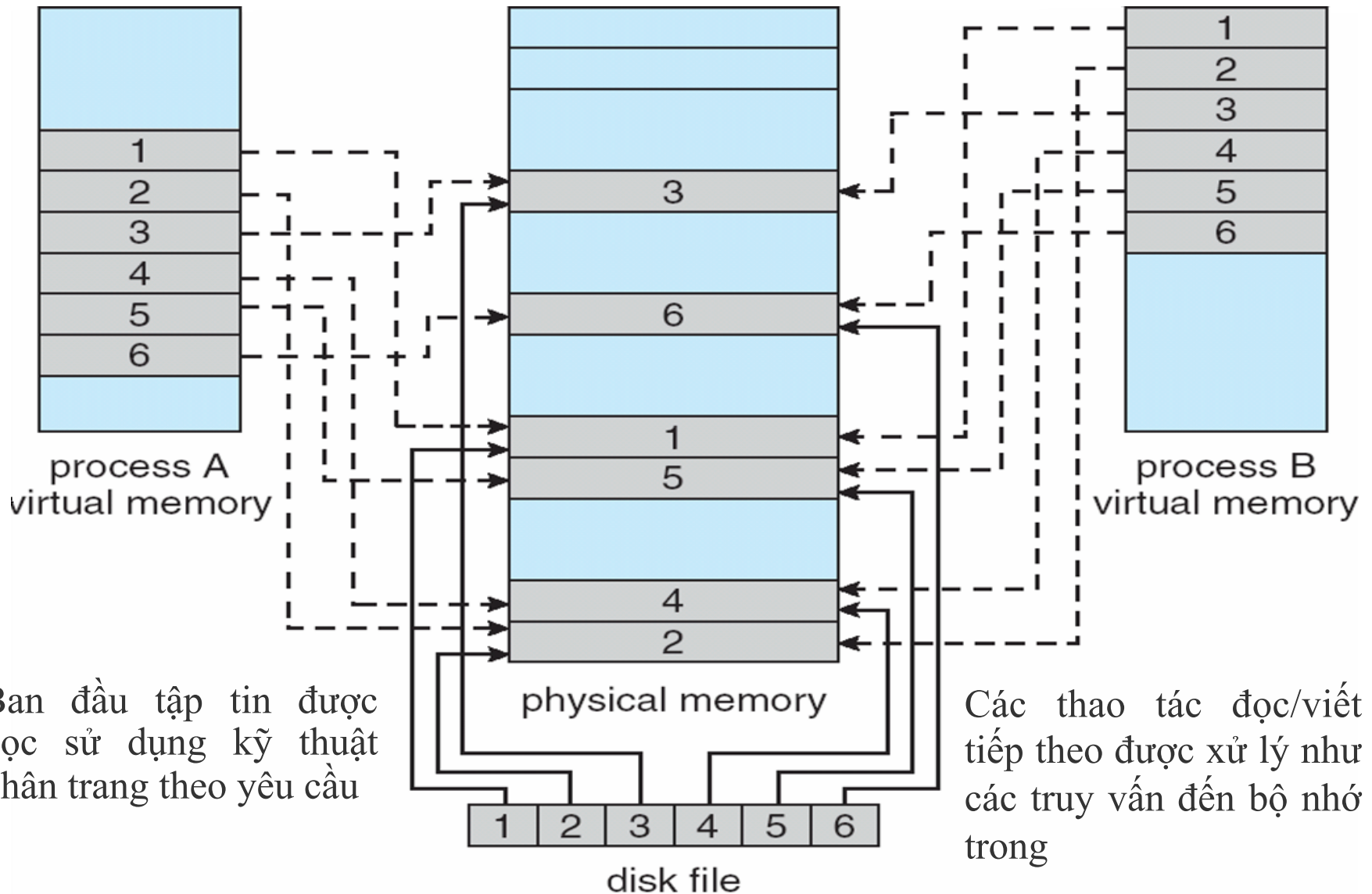


- Khi tỉ suất page fault tăng mà không có frame rỗi nào thì lựa chọn một tiến trình để dừng

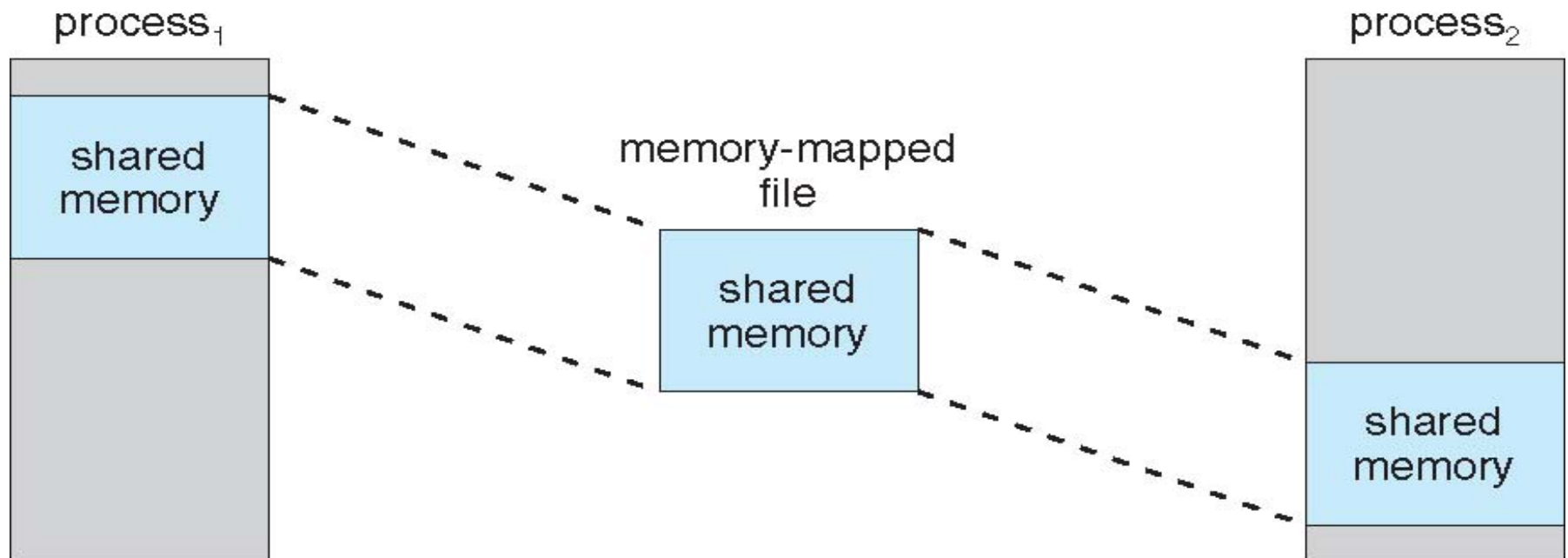
Phần 8: Bộ nhớ ảo

8.7 Tập tin ánh xạ bộ nhớ

Ví dụ tập tin ánh xạ bộ nhớ



Ví dụ tin ảnh xạ bộ nhớ chia sẻ trong Windows

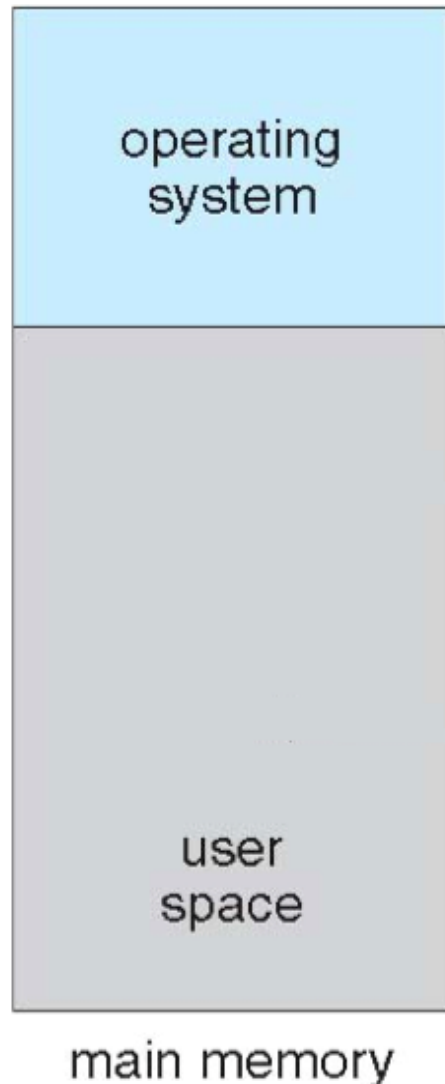


Phần 8: Bộ nhớ ảo

8.8 Cấp phát bộ nhớ cho HĐH

Đặc điểm của cấp phát bộ nhớ cho HĐH

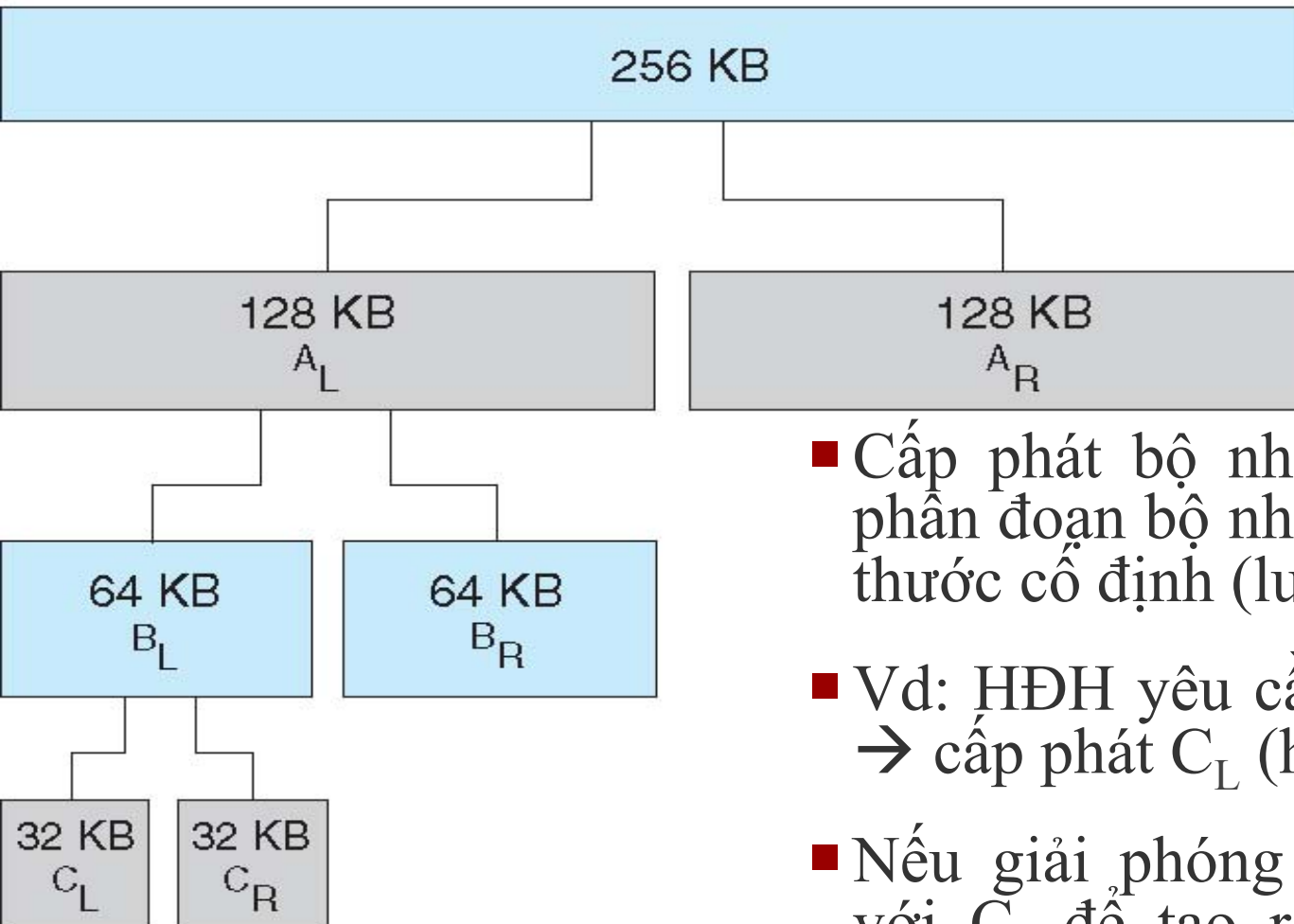
37



- Cách thức khác so với cấp phát bộ nhớ cho chương trình người dùng
 - Nhân HĐH thường yêu cầu bộ nhớ với các kích thước khác nhau
 - Một số thiết bị phần cứng cần bộ nhớ HĐH liên tục

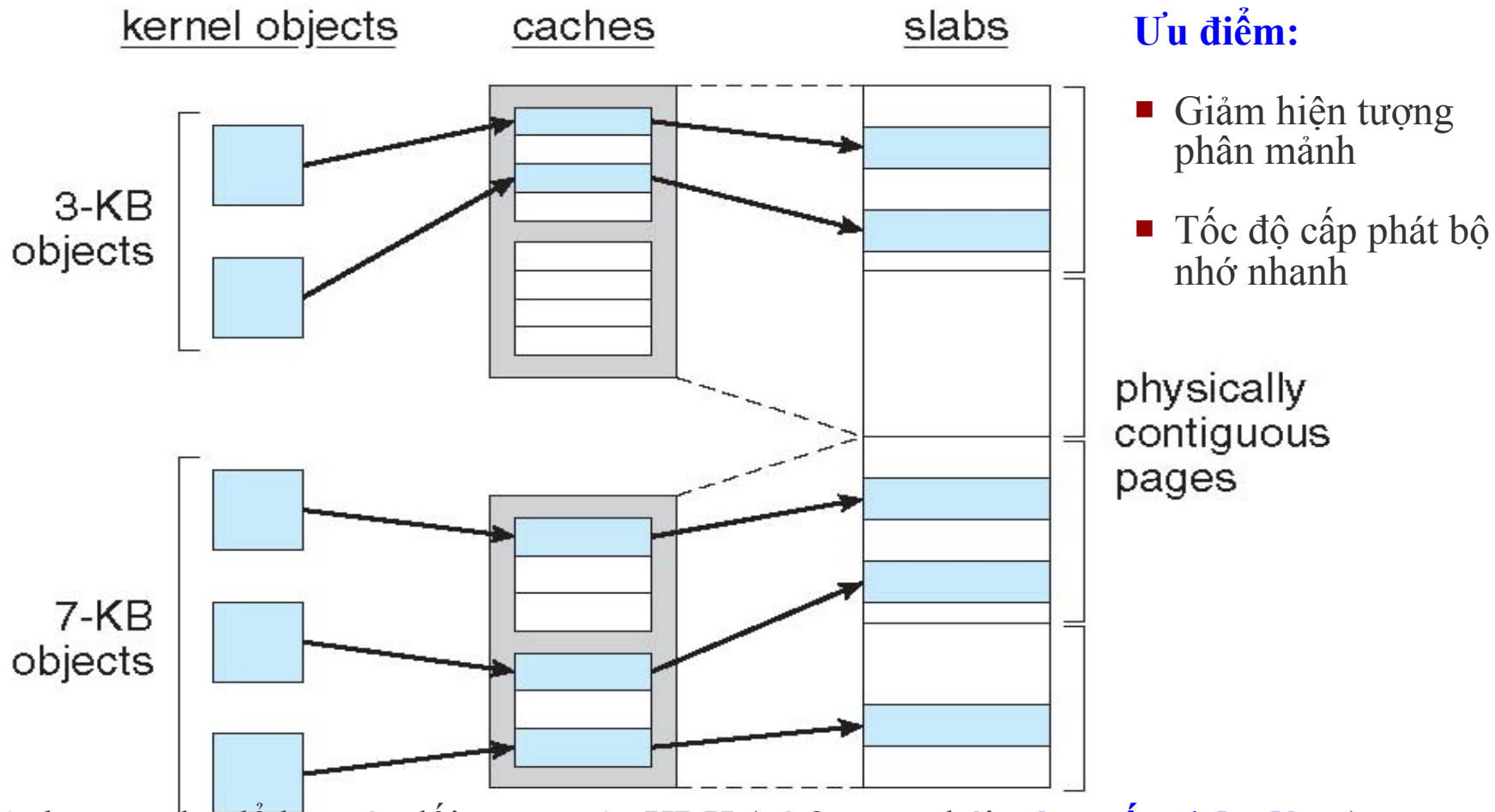
Bộ cấp phát hệ thống Buddy

physically contiguous pages



- Cấp phát bộ nhớ dưới dạng các phân đoạn bộ nhớ liên tục có kích thước cố định (luỹ thừa của 2)
- Vd: HĐH yêu cầu 21 KB bộ nhớ → cấp phát C_L (hoặc C_R)
- Nếu giải phóng C_L thì hợp nhất với C_R để tạo ra phân đoạn lớn hơn

Bộ cấp phát slab



Ưu điểm:

- Giảm hiện tượng phân mảnh
- Tốc độ cấp phát bộ nhớ nhanh

- Sử dụng cache để lưu các đối tượng của HĐH (có 2 trạng thái: **còn trống/ đã dùng**)
- Số lượng đối tượng trên mỗi cache phụ thuộc vào kích thước của các slab tương ứng

Phần 8: Bộ nhớ ảo

8.9 Một số vấn đề khác cần quan tâm

Một số vấn đề khác cần quan tâm với kỹ thuật phân trang

41

■ Phân trang trước

- Giảm số page fault bằng cách đưa sẵn một số trang vào bộ nhớ trước khi tiến trình dùng chúng
- Có thể lãng phí vào/ra dữ liệu và bộ nhớ

■ Kích thước trang

- Phụ thuộc thiết kế HĐH
- Lũy thừa của 2

■ Tổng dung lượng bộ nhớ truy cập được từ TLB

$$=(\text{Kích thước TLB}) \times (\text{Kích thước trang})$$

- Tất cả các trang trong tập đang làm việc nên được lưu trong TLB

Một số vấn đề khác cần quan tâm với kỹ thuật phân trang

42

■ Cấu trúc chương trình

- `Int[128,128] data;`
- Mỗi hàng được lưu trong một trang
- Program 1

```
for (j = 0; j < 128; j++)  
    for (i = 0; i < 128; i++)  
        data[i,j] = 0;
```

128 x 128 = 16,384 page fault

■ Program 2

```
for (i = 0; i < 128; i++)  
    for (j = 0; j < 128; j++)  
        data[i,j] = 0;
```

128 page fault

Bài tập

- Hệ điều hành có 256 KB bộ nhớ trong, ban đầu trống hoàn toàn. Các tiến trình yêu cầu sử dụng bộ nhớ với kích thước 5 KB, 25 KB, 35 KB, 20 KB.
- Hãy mô tả cách hệ thống Buddy đáp ứng mỗi yêu cầu, vẽ sơ đồ bộ nhớ tương ứng.
- Sau khi cấp phát hết các yêu cầu trên, hệ thống sẽ làm gì nếu tiến trình với yêu cầu bộ nhớ 25 KB kết thúc và giải phóng bộ nhớ ?